

Searching Binary Embeddings

Bitplanes and Prefix Backward Walk

12 September 2026

Abstract

This report explores two ways to improve the search stage described in Exa’s vector-database article: Bitplanes and Backward Walk. Backward Walk now means prefix relaxation: start with a specific prefix and remove its last bit when more candidates are needed. The earlier exact-key method is called C_{key} . Its results are historical; the prefix method described here has not yet been implemented or benchmarked.

1 Introduction

1.1 Exa method and end to end pipeline

Let’s first understand the “original” pipeline that I am trying to improve upon.

Note that the architecture I am referring to here is based on the pipeline described in Exa’s article [1]. I am aware that Exa’s actual production system may have changed or improved since then, but for the purpose of this report, let’s treat the architecture described in that article as the original baseline that we compare against.

In this pipeline, document embeddings are first shortened using Matryoshka representations, converted into binary signs, and assigned to clusters. At query time, the query is encoded, a subset of nearby clusters is selected, and the documents inside those clusters are scored using the floating-point query. The top candidates can then be reranked using the original uncompressed vectors.

The main part that I am trying to improve in this report is the search stage after cluster selection. The two new approaches introduced later, Bitplanes and Backward Walk, replace this step while keeping the rest of the pipeline largely the same.

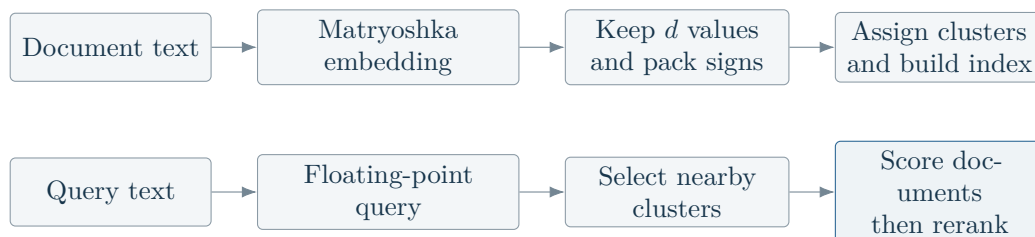


Figure 1: The baseline pipeline; B and C change document search inside the selected clusters.

1.2 Idea background

When I took a closer look at the pipeline, one part that particularly stood out to me was the comparison between the floating-point query and the binary-quantized document. Consider a simple example:

$$q = [0.7, 0.5, -0.4, -0.6, 0.3], \quad \text{document} = 11000.$$

Suppose a document is represented using binary signs, where 1 means +1 and 0 means −1. The score is calculated by taking the dot product between the query values and the document signs. So for 11000, the score is

$$0.7(+1) + 0.5(+1) + (-0.4)(-1) + (-0.6)(-1) + 0.3(-1) = 1.9.$$

Now, something interesting follows from this. Just by looking at the signs of the query, we already know what the ideal binary document would look like. For every positive query value, we would prefer the corresponding document bit to be 1, and for every negative query value, we would prefer it to be 0.

Therefore, for the query above, the ideal binary pattern is 11001. This document would achieve the maximum possible score:

$$0.7 + 0.5 + 0.4 + 0.6 + 0.3 = 2.5.$$

From here, we can think about every other document in terms of how far it is from this ideal pattern. More importantly, not every mismatch has the same cost. If we mismatch the first coordinate, where $|q_1| = 0.7$, the score decreases by 1.4. But if we mismatch the last coordinate, where $|q_5| = 0.3$, the score only decreases by 0.6.

$$Q = \sum_i |q_i|, \quad p = \sum_{i: b_i \neq \text{sign}(q_i)} |q_i|, \quad S(q, b) = Q - 2p. \quad (1)$$

Q is the score of the ideal binary pattern. p adds the absolute query values at mismatched coordinates. In this formula, b_i is the decoded sign (+1 or -1); the printed codes use 1 and 0. This observation is the main idea behind the approaches explored in this report.

Instead of treating every binary document equally and scoring all of them one by one, can we use the query itself to guide the search toward documents that are closer to its ideal binary pattern, while avoiding unnecessary full-score calculations?

1.3 Exploring new methods

From the previous section, we know two useful things before we even start searching: we know what the ideal binary document looks like from the signs of the query, and we know that some bits matter more than others, based on the magnitude $|q_i|$. The next question is therefore: can we build an index that uses this information directly, instead of treating every document inside the selected clusters equally?

This led me to explore two approaches.

The first, Bitplanes, starts from a large set of candidate documents and progressively narrows it down using the query.

The second, Backward Walk, approaches the problem from the opposite direction: it starts from a very specific binary pattern and progressively relaxes the constraints until enough useful candidates are found.

1.3.1 Bitplanes

Now that we know what the ideal document looks like based on the query, one idea is to build a data structure that makes filtering by individual bits very cheap. Normally, binary documents are stored row by row. Instead, we can additionally store an inverted representation of the same bits:

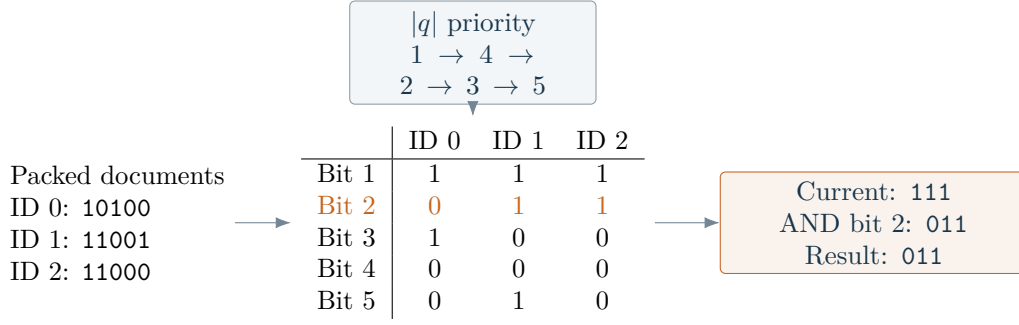


Figure 2: The same document signs, stored in an extra layout for bitwise filtering.

Each row above is a bitplane: one bit position across many documents. Suppose the query is $q = [0.7, 0.5, -0.4, -0.6, 0.3]$. From the magnitude of the query values, we prioritize the bits as 1, 4, 2, 3, 5, because $0.7 > 0.6 > 0.5 > 0.4 > 0.3$. Now suppose our current candidate mask is 111, meaning that IDs 0, 1 and 2 are all still possible candidates.

For bit 2, the query prefers 1. The corresponding bitplane is 011, so keeping the documents that match the query is simply $111 \& 011 = 011$. We immediately reduce the candidate set to IDs 1 and 2. The earlier splits on bits 1 and 4 keep all three rows in this example.

This is attractive because the filtering itself consists mostly of bitwise operations. The Python below shows a simplified first version. Here, a cluster provides its bitplanes and rows. Masks use Python integers to keep the example short; the measured C++ code uses fixed-width word arrays. `full_score` writes out the dot product; the cost calculations later describe the C++ lookup-table scorer, not Python operation costs. `TopK` keeps the best K scores, breaking ties by document ID, and `rows_in_mask` yields the selected rows without copying them:

```
def full_score(query, code):
    return sum(q if bit else -q for q, bit in zip(query, code))

def greedy_search(query, cluster, k, leaf_size):
    order = sorted(range(len(query)), key=lambda i: -abs(query[i]))
    active = cluster.full_mask
    for bit in order:
        plane = cluster.planes[bit]
        preferred = plane if query[bit] >= 0 else ~plane
        active &= preferred
        if active.bit_count() <= leaf_size:
            break

    best = TopK(k)
    for row_id, code in cluster.rows_in_mask(active):
        best.add(row_id, full_score(query, code))
    return best.results()
```

PROBLEM 1: We might miss the actual best document.

Consider $q = [0.6, 0.5, 0.4, 0.3]$. The ideal binary document is 1111. Suppose the database only contains document A: 0111, and document B: 1000. If we greedily start with the first bit, the query prefers 1. We would therefore keep document B and throw away document A. But their complete scores are:

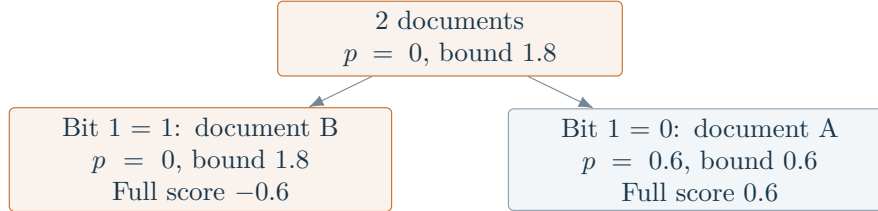
$$S(A) = -0.6 + 0.5 + 0.4 + 0.3 = 0.6, \quad S(B) = 0.6 - 0.5 - 0.4 - 0.3 = -0.6.$$

So document A is actually better. The problem is that matching one important bit does not guarantee that the remaining bits will also match.

Solution: branching.

Instead of discarding the alternative, we keep both branches.

The branch matching the preferred bit receives no additional mismatch penalty. The alternative branch is still kept, but its penalty increases by $|q_i|$. We then explore the branches in order of smallest accumulated penalty. This gives us a search tree rather than one greedy path.



After scoring B, the other bound is $0.6 > -0.6$.
We must explore A; the first choice was not enough.

Figure 3: Keep the alternative until its score bound proves it cannot improve the result.

Stopping condition.

If a branch has already accumulated mismatch penalty p , any document below it must have penalty at least p . Therefore, $Q - 2p$ is an upper bound on the score of every document inside that branch. For example, suppose $Q = 2.5$ and we are looking for the top two results. We have already found scores 2.5 and 1.9.

An unexplored branch with $p = 0.5$ can score at most $2.5 - 2(0.5) = 1.5$. Since $1.5 < 1.9$, nothing inside that branch can enter our current top two.

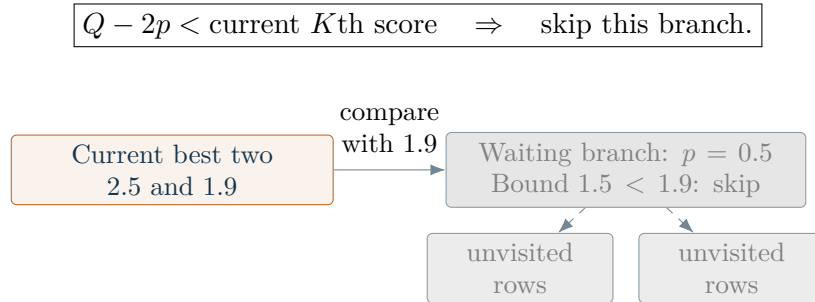


Figure 4: The whole gray subtree can be skipped because even its best possible score is too low.

If the best remaining branch in the queue already fails this test, then every other branch is worse and the entire search can stop. Equal scores still follow the document-ID tie rule.

```

from heapq import heappop, heappush
from itertools import count

def bitplane_search(query, clusters, k, leaf_size):
    order = sorted(range(len(query)), key=lambda i: -abs(query[i]))
    ideal_score = sum(abs(q) for q in query)
    sequence = count()
    queue = []
    for cluster in clusters:
        if cluster.full_mask:
            heappush(queue, (0, next(sequence), 0,
                             cluster.full_mask, cluster))

    best = TopK(k)
    while queue:
        penalty, _, depth, active, cluster = heappop(queue)
        upper_bound = ideal_score - 2 * penalty
        if best.full() and upper_bound < best.worst_score():
            break
        if active.bit_count() <= leaf_size or depth == len(query):
            for row_id, code in cluster.rows_in_mask(active):
                best.add(row_id, full_score(query, code))
            continue

        bit = order[depth]
        plane = cluster.planes[bit]
        preferred = plane if query[bit] >= 0 else ~plane
        matching = active & preferred
        # Keep the other half too; its remaining bits may be better.
        children = ((matching, 0), (active ^ matching, abs(query[bit])))
        for mask, extra_penalty in children:
            if mask:
                heappush(queue, (penalty + extra_penalty,
                                next(sequence), depth + 1, mask, cluster))

    return best.results()

```

PROBLEM 2: The bitmap can still be large.

There is another issue. A bitwise AND sounds almost free, but it is not literally one operation when the cluster contains thousands or millions of documents. Suppose one cluster contains 6400 documents. With 64-bit machine words, one bitplane contains $6400/64 = 100$ machine words. So one split requires us to loop over those 100 words.

Even worse, suppose after several splits only 80 documents remain. The physical mask can still be 100 words wide, so the next split may still need to process those same 100 words.

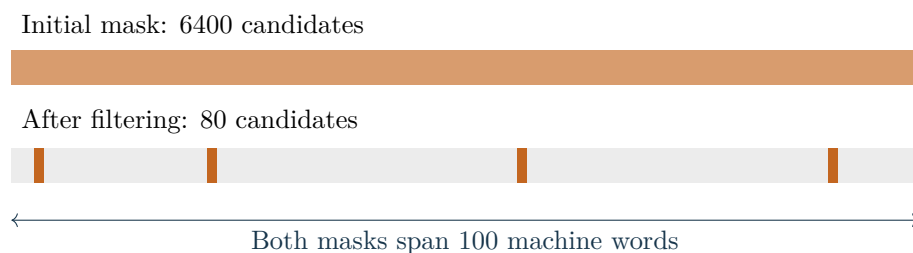


Figure 5: Fewer active document positions do not make this dense mask physically shorter.

Bitmap traversal cost. In general, for a cluster containing n documents, one dense bitplane

operation costs roughly $O(\lceil n/64 \rceil)$ word visits. If we visit J split nodes, this becomes $O(J \lceil n/64 \rceil)$, before counting queue operations and final document scoring. This was one of the important findings from the earlier experiments: reducing the number of fully scored documents does not automatically mean reducing total query time, because the bitmap traversal itself can become expensive.

Solution 1: Parameter tuning. The question is finding the point where the time saved by avoiding scores is worth more than the additional bitmap traversal. We can tune cluster size during coarse routing, the number of clusters probed, how deep we split, leaf size, and when we switch back to ordinary full scoring. These are options to test, not a guarantee that tuning will make Bitplanes faster.

Solution 2: Backward Walk. However, thinking about this cost led me to a second idea. Bitplanes starts with a large set of documents and repeatedly narrows it down. What if we instead did the opposite? If this approach is bottom-up, the walk backwards can be described as top-down. Here those names describe broad-to-specific versus specific-to-broad search. In an actual prefix tree, the backward walk moves toward the root as it removes bits.

1.3.2 Backward Walk

Backward Walk builds on the same observation: given a query, we already know what its ideal binary document should look like. However, instead of starting with thousands of documents and filtering them using bitplanes, we try to start directly from a small bucket of documents that already looks similar to the query.

Suppose the binary query begins with $101101001011\dots$. Using all 256 bits as a lookup key can be far too specific. Under a balanced independent-bit model, an exact match is extremely unlikely; real occupancy must be measured. Instead, suppose we choose a prefix length x . For example, the first 16 bits: 1011010010110101 . During index construction, documents sharing this prefix are placed in the same bucket or contiguous range.

At query time, we directly look up this prefix. If it contains enough useful documents, we score them using the complete 256-bit score. If it is empty, or contains too few candidates, we remove one constraint:

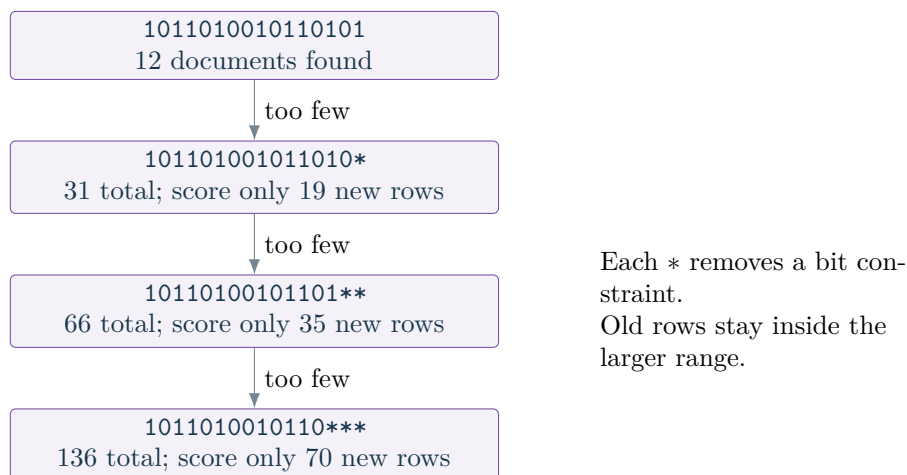


Figure 6: Illustrative counts: $12 + 19 + 35 + 70 = 136$ distinct rows scored, not $12 + 31 + 66 + 136$.

Note that the expected document count already grows exponentially as we remove prefix bits. If the bits are independent and equally likely to be 0 or 1, each removed bit doubles that count.

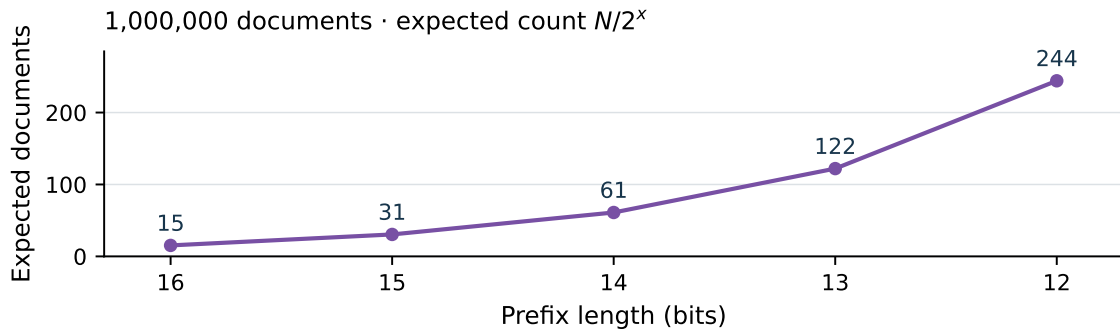


Figure 7: Expected counts for one global cluster; labels are rounded. This estimates rows, not recall.

So we can widen one bit at a time and score only the newly exposed documents, as in Figure 6. Real prefix counts can differ from this simple model.

Lookup and widening.

A trie or rows sorted by prefix can support the lookup. The proposed first implementation uses sorted keys and row ranges, with no document graph. Each broader prefix contains the previous range. Subtracting the old range means processing at most two new slices: one on either side. The query keeps the original bit order. It does not sort the prefix bits by $|q_i|$, and it does not enumerate arbitrary flipped keys.

```
def backward_walk(query, clusters, k, start_depth, candidate_target):
    signs = tuple(q >= 0 for q in query)
    previous = [None] * len(clusters)
    best = TopK(k)
    scored = 0
    for depth in range(start_depth, -1, -1):
        for index, cluster in enumerate(clusters):
            left, right = cluster.prefix_range(signs[:depth])
            if previous[index] is None:
                new_ranges = [(left, right)]
            else:
                old_left, old_right = previous[index]
                # Only the new rows beside the old range need scores.
                new_ranges = [(left, old_left), (old_right, right)]
            for start, stop in new_ranges:
                for row_id, code in cluster.rows_in_range(start, stop):
                    best.add(row_id, full_score(query, code))
                scored += stop - start
            previous[index] = (left, right)

        # Finish this depth in every cluster before testing the target.
        if candidate_target > 0 and scored >= candidate_target:
            break
    return best.results()
```

Here, `prefix_range` returns the two positions enclosing a prefix in sorted rows. `rows_in_range` yields those rows without copying them. The Python illustrates the search steps; it is not a new benchmark implementation.

This proposed stopping rule completes a whole depth across all opened clusters before checking the candidate target. It can return more candidates than the target. A target of zero disables early stopping and continues to depth zero. Stopping simply because we have collected, for example, 100 candidates is an approximate stopping rule. It does not mathematically prove that a better document does not exist outside the current prefix.

Therefore, recall still needs to be measured against the same exact reference results. An exact early-stop version would require a valid bound on the documents outside the visited prefixes. Reaching depth zero scores every document in the opened clusters, once. That matches the original scan’s local result, though not necessarily its time.

Relationship to HNSW.

At first glance, this approach may sound somewhat similar to HNSW because both methods use a hierarchy to progressively change the search space. However, the hierarchy is being used differently. HNSW builds a proximity graph. During a query, it begins from sparse upper layers, navigates toward promising neighbors, and descends toward the denser bottom layer [2].

Backward Walk does not need to navigate between neighboring document vectors. The query itself directly tells us where to begin: query $\rightarrow$ binary prefix $\rightarrow$ prefix lookup. HNSW is not a root-to-leaf tree and is not restricted to nonbinary data. The relevant difference is graph navigation versus a prefix lookup.

	HNSW	Backward Walk
Index	Neighbor graph	Prefix keys and ranges
Query start	Upper graph layer	Query’s own prefix
Movement	Neighbor to neighbor	Specific to broader prefix
Insertion	Find neighbors; update edges	Compute prefix; update storage

Potential advantage 1: index construction and updates.

One possible advantage that I would like to investigate further is the cost of adding new documents. For HNSW, inserting a new vector generally requires searching the existing graph to find suitable neighbors and then creating or updating graph connections. For Backward Walk, the conceptual insertion process is simpler: document $\rightarrow$ embedding $\rightarrow$ binary signs $\rightarrow$ prefix key $\rightarrow$ corresponding bucket.

There is no nearest-neighbor graph that needs to be constructed for each new document. So Backward Walk may be attractive for a search index that changes frequently. I would frame this as a hypothesis to verify, because the actual insertion cost still depends on how the buckets, ranges and index are implemented.

In particular, inserting into a sorted array can move many rows or require a rebuild. The first planned version is static; it does not establish cheap online updates.

Potential advantage 2: Matryoshka prefixes give the relaxation some meaning.

A Matryoshka model is trained so that shorter prefixes of the embedding remain useful representations at its trained lengths [3]. This gives us a reason to investigate shorter prefixes. It does not mean that the first coordinate is always more important than the second, or that the nearest neighbor under 16 dimensions must remain the nearest neighbor under 15 dimensions.

The actual recall of each prefix still has to be measured. Binary sign agreement is also different from similarity under the original float prefix. The Nomic model card reports float-prefix results at 64, 128, 256, 512 and 768 dimensions [4]. It does not establish high recall for every 16-to-15-bit sign-prefix relaxation.

2 Preliminary calculations

The collection contains $N = 1,000,000$ documents. The same 200 query vectors, 256-bit codes and reference answers are used throughout. Each method may choose its own cluster count C , probe count P and local settings. A probe opens one cluster. The result count is $K = 100$; d denotes the number of document bits.

2.1 Time complexity

One example for all three methods. Open one cluster containing these three rows. Return the best two results: $N = 3$, $C = P = 1$, $d = 5$, $K = 2$.

$$q = [0.7, 0.5, -0.4, -0.6, 0.3], \quad \text{ideal signs: } 11001.$$

ID	Document bits	Full score	Calculation
0	10100	0.1	$0.7 - 0.5 - 0.4 + 0.6 - 0.3$
1	11001	2.5	$0.7 + 0.5 + 0.4 + 0.6 + 0.3$
2	11000	1.9	$0.7 + 0.5 + 0.4 + 0.6 - 0.3$

These are teaching rows, not a change to the benchmark dataset. Bits are shown in position order, left to right. Scores are rounded here. For larger collections, $n = N/C$ estimates rows per cluster if clusters are equal-sized.

2.1.1 Original method

Step 1: open the cluster and prepare the query.

Open cluster 0: [ID 0, ID 1, ID 2]. One cluster lookup here. Following Exa’s four-sign table method [1], split the five query values into $[0.7, 0.5, -0.4, -0.6]$ and $[0.3, 0, 0, 0]$. Each group has 16 sign patterns: $2 \times 16 = 32$ table entries. The final group is padded.

The code checks four slots per entry: $32 \times 4 = 128$ slot visits. Table setup is $O(d)$ for a fixed group width of four. Cluster selection costs T_{route} .

Step 2: score each document.

$$\text{ID } 0 \longrightarrow 0.1 \quad \text{ID } 1 \longrightarrow 2.5 \quad \text{ID } 2 \longrightarrow 1.9$$

Loop over three rows; each reads two score-table entries: $3 \times 2 = 6$ terms. For F_A rows, this is $F_A \lceil d/4 \rceil$ terms, or $O(F_A d)$ work.

Step 3: keep the best two.

$$[0:0.1] \longrightarrow [1:2.5, 0:0.1] \longrightarrow \boxed{[1:2.5, 2:1.9]}.$$

Offer three scores; keep at most two. The display above is sorted for readability. Keeping K results costs up to $O(F_A \log K)$; final sorting costs $O(K \log K)$.

Total. One cluster lookup, 32 table entries built, six table terms read and three scores offered. These are different kinds of work, so we do not add them as equal CPU operations.

$$\boxed{T_A \approx T_0 + \frac{PN}{C} a_A.}$$

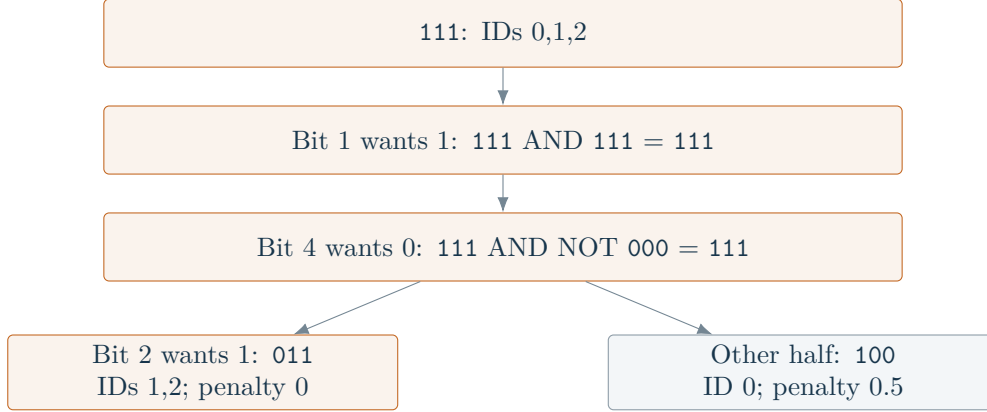
T_0 includes cluster selection and table setup. a_A is average time per document, including its score and result update. For fixed C, P, d, K , scan work grows as $O(N)$.

2.1.2 Bitplanes

Same input: $q = [0.7, 0.5, -0.4, -0.6, 0.3]$; rows 10100, 11001, 11000; best two results. Stop splitting when at most two rows remain. Keep all other work limits unlimited.

Step 1: open, prepare and order the bits.

Open the same cluster and build the same 32 score-table entries as A. Sort five query positions by absolute value: $1 (0.7) \rightarrow 4 (0.6) \rightarrow 2 (0.5) \rightarrow 3 (0.4) \rightarrow 5 (0.3)$. Sorting costs $O(d \log d)$. One pass sums the five absolute values: $Q = 2.5$, costing $O(d)$. Write one word for mask 111: all rows are active. Initial masks cost $O(PW)$ word writes.

Step 2: split the mask.

The first two splits remove nothing. The third separates ID 0 from IDs 1 and 2. All three masks still have three positions. Each occupies one 64-bit word. Three splits visit $3 \times 1 = 3$ word positions. With J splits and mask width W , this costs $O(JW)$, where $W = \lceil n/64 \rceil$.

Step 3: score the two remaining rows.

Mask 011 fits the leaf limit. Read one mask word, then score IDs 1 and 2. Two scores use $2 \times 2 = 4$ table terms. Best results: $[1: 2.5, 2: 1.9]$. Reading L final masks costs $O(LW)$; scoring F_B rows costs $O(F_B d)$. Result updates have the same $O(F_B \log K)$ bound as A.

Step 4: check the waiting branch and stop.

Its best possible score is $2.5 - 2(0.5) = 1.5$, below the retained score 1.9. Skip ID 0. One final bound check is $O(1)$ once the query sum and penalty are known. Four candidate sets were processed: three splits and one leaf. Including the P initial sets, queue work is bounded by $O((P + V) \log(P + V + 1))$.

Total. Same table setup as A; sum five weights; sort five positions; write one initial mask; visit three split words and one final-mask word; score two rows; skip one row. Fewer scores alone do not prove lower time.

$$T_B \approx T_0 + PS \left\lceil \frac{N}{64C} \right\rceil b + f_B \frac{PN}{C} a_B + T_{\text{other},B}.$$

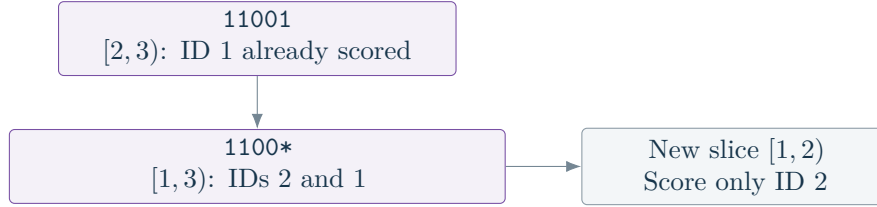
S is average splits per opened cluster; f_B is the scored fraction; b is time per split-word visit. Here $S = 3$, $f_B = 2/3$. Other time includes sorting, queues and final-mask reads. A wide mask still costs time after many rows are removed.

2.1.3 Backward Walk: prefix relaxation

This is the proposed prefix method. The steps below are arithmetic examples, not measurements of an implemented index. Use the same query and three rows as A and B. Start at depth $x = 5$ and stop after scoring at least two candidates, after completing that depth across the selected clusters.

Step 1: find the full prefix.

Query signs: 11001. Sort the stored five-bit keys in position order: 10100, 11000, 11001. Two binary searches find the start and end of prefix 11001: range $[2, 3)$ contains ID 1. Score ID 1 using all five values: 2.5. Only one candidate has been scored, so continue.

Step 2: remove the last constraint.

The parent range contains the old range. Score only the new row, ID 2, which has score 1.9. Two distinct candidates have now been scored. Stop and return IDs 1 and 2. This happens to recover both reference IDs in this example. A candidate count of two does not guarantee top-two recall on other data.

Step 3: count the work.

Two prefix depths, one cluster, and two binary searches per range: four binary-search calls. Each call over three keys uses at most two key comparisons: at most eight comparisons in this example, plus range checks. Score two rows, once each: four score-table terms. Query table setup and result updates still cost time.

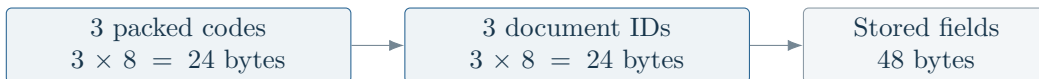
$$T_C \approx T_0 + 2PL \log_2(n+1)t_b + F_C a_C + T_{\text{ranges}}.$$

P is opened clusters; $L = x - \ell + 1$ is visited depths down to final depth ℓ ; $n = N/C$ assumes equal cluster sizes. t_b estimates time per key comparison. F_C is distinct rows scored. T_{ranges} includes prefix updates and new-slice checks. For unequal clusters use $2L \sum_{c \text{ opened}} \log_2(n_c + 1)$ as the comparison-count estimate. Constants and cache effects still need measurements.

In a static sorted-key index, range lookup is $O(PL \log(n+1))$. Full scoring is $O(F_C d)$, plus top- K work. A trie would have a different lookup cost. Under independent balanced bits at a fixed final depth, $\mathbb{E}[F_\ell] = Pn/2^\ell$. This is not a recall estimate or a prediction of the adaptive stopping depth.

2.2 Memory complexity

Same example: three documents, five bits each, one cluster, two results. We count bytes of data first. Full process RAM also includes unused capacity, queue records, hash-table overhead, Python, and other runtime memory.

2.2.1 Original method**Step 1: store the codes and IDs before search.**

Each five-bit row uses one eight-byte word. Packing all 15 bits across row boundaries would use a different layout; this implementation stores whole words per row. Formula: $M_{\text{rows}} = N(8 \lceil d/64 \rceil + 8)$ bytes. Size is $O(N \lceil d/64 \rceil + N)$. Cluster data is extra. For example, C float32 centroids of width d_R use $4Cd_R$ bytes. This one-cluster example needs no centroid search.

Step 2: hold the query, score table and current results during search.

$$\boxed{5 \times 4 = 20} \quad + \quad \boxed{32 \times 8 = 256} \quad + \quad \boxed{2 \times (8 + 8) = 32} \quad = \quad 308 \text{ bytes.}$$

Left to right: float32 query, double-precision table entries, result IDs and scores. The result buffer holds two entries even though A checks three documents. It does not need a stored score for every document. General sizes: $4d + 128 \lceil d/4 \rceil + 16K$ bytes, or $O(d + K)$ for these buffers.

Total counted fields: 48 bytes stored + 308 query bytes = 356 bytes. This is not total process RAM; cluster records and allocation overhead remain additional.

2.2.2 Bitplanes

Step 1: keep A's rows and add the bitplanes.

	ID 0	ID 1	ID 2	Packed storage
Bit 1	1	1	1	8 bytes
Bit 2	0	1	1	8 bytes
Bit 3	1	0	0	8 bytes
Bit 4	0	0	0	8 bytes
Bit 5	0	1	0	8 bytes

Each plane has three useful bits but occupies one word. Five planes add $5 \times 8 = 40$ bytes. Stored fields: $48 + 40 = 88$ bytes. Formula: $M_{\text{planes}} = 8d \sum_c \lceil n_c/64 \rceil$ bytes. Include padding in every cluster.

Step 2: hold masks while splitting.

$$\underbrace{111}_{\text{parent: 8 bytes}} \longrightarrow \underbrace{011}_{\text{child: 8 bytes}} + \underbrace{100}_{\text{child: 8 bytes}} .$$

The parent exists while both children are built: $3 \times 8 = 24$ live mask bytes at the peak. All three masks keep their full width, even if one child is empty. Sorting five positions adds $5 \times 8 = 40$ bytes on this 64-bit machine. Counted query data: A's $308 + 24 + 40 = 372$ bytes. Queue records are still extra. For mask width $W_c = \lceil n_c/64 \rceil$ and $A_c(t)$ live masks, peak mask data is $8 \max_t \sum_c A_c(t) W_c$ bytes. Waiting branches count too.

Total counted fields: 88 bytes stored + 372 query bytes = 460 bytes. Stored size is $O(N \lceil d/64 \rceil + N + d \sum_c W_c)$; query masks grow with the number and width of live branches, not just the number of remaining document IDs.

2.2.3 Backward Walk: proposed sorted-key storage

Step 1: store rows in prefix order.

Position	Full binary document	ID
0	10100	0
1	11000	2
2	11001	1

Keep one copy of full packed codes and IDs: $3 \times (8 + 8) = 48$ bytes of fields for the five-bit example. The proposed first layout also stores one four-byte prefix key per row, for maximum prefix widths up to 32 bits: $3 \times 4 = 12$ bytes. Counted index fields: $48 + 12 = 60$ bytes. No separate row list is stored at every prefix depth; binary search finds each range in the same sorted key array.

$$M_{C,\text{fields}} = N(8 \lceil d/64 \rceil + 8 + 4) + M_{\text{routing}}.$$

With one million 256-bit rows, those code, ID and key fields total 44 MB, before routing and allocation overhead. This is a proposed layout calculation, not a measured memory result. Wider-than-32-bit stored prefixes would need a larger key type.

Step 2: hold range boundaries during search.

Keep the previous range for each opened cluster: two eight-byte offsets, or $16P$ bytes of fields. Hold the current range and process only its new slices. This avoids a bitmap over all documents and a list of all visited document IDs. Query vector, score table and top- K buffers remain. Their sizes follow A; added range bookkeeping grows as $O(P)$, not $O(N)$.

Full process RAM includes array capacity, routing, temporary build copies and the runtime. It must be measured after implementation.

Total. Proposed persistent space is $O(N \lceil d/64 \rceil + N)$ plus routing. Query work space is $O(P + d + K)$ for this layout; index construction and online insertion costs remain separate.

2.3 Recall estimation

Reference IDs are the exact best results under the full binary score. In our small example they are [1, 2]. All three worked examples return both: $2/2 = 100\%$ recall; the prefix example is not an implemented measurement. In the benchmark, each query has $K = 100$ reference IDs. Returning 99 gives 99% recall. This checks the ranking, not whether the text answers a person's question.

Let G_q be reference IDs and $O_{a,q}$ returned IDs. Across the same $n_q = 200$ queries,

$$\overline{R}_a = \frac{1}{n_q K} \sum_q |G_q \cap O_{a,q}|.$$

2.3.1 Original method

Let g_q reference IDs be in the opened clusters. A scores all opened rows, so recall is g_q/K .

2.3.2 Bitplanes

If B returns fraction $r_{B,q}$ of those IDs, recall is $(g_q/K)r_{B,q}$. For example, $(8/10)(6/8) = 60\%$. Safe, unlimited local search gives $r_{B,q} = 1$.

2.3.3 Backward Walk

Let $D_{q,\ell}$ be rows in the opened clusters matching the query's first ℓ bits. After widening to ℓ , every row in that set has been scored exactly once. Recall is $|G_q \cap D_{q,\ell}|/K$ when the full top- K over those rows is returned. Measure the intersection; row count alone is insufficient.

At depth zero, $D_{q,0}$ is every row in the opened clusters, so C matches A's local recall. A candidate target can stop earlier and miss reference IDs. The first planned version has no early exactness claim. A valid bound on all unseen rows would be required to make one.

2.4 Summary

Each formula uses its method's own opened clusters and local work. Let M_0 be allocated row and router storage, Q_0 common query memory, and Q_B extra branch memory. C adds a four-byte key per row and per-query range records; its allocation overhead is still to be measured.

Time complexity

Method	Approximate query time
Original method	$T_0 + (PN/C)a_A$
Bitplanes	$T_0 + PS \lceil N/(64C) \rceil b + f_B(PN/C)a_B + T_{\text{other,B}}$
Backward Walk	$T_0 + 2PL \log_2(n+1)t_b + F_C a_C + T_{\text{ranges}}$ (proposed sorted-key layout)

The symbols and loop counts are defined in the steps above. Use actual cluster sizes and key counts for the measured comparison. Each cost belongs in one term only.

Memory complexity

Method	Stored index	Temporary query memory
Original method	M_0	Q_0
Bitplanes	$M_0 + M_{\text{planes,allocated}}$	$Q_0 + Q_B$
Backward Walk	$M_0 + 4N$ (prefix keys)	$Q_0 + O(P)$ (range records)

The process, including these arrays and the running program, must fit 32 GB. A peak that includes construction is reported as such, not as index size.

Recall estimation

Method	Per-query recall	Local exactness condition
Original method	g_q/K	Score all opened documents
Bitplanes	$(g_q/K)r_{B,q}$	Complete search with safe bounds
Backward Walk	$ G_q \cap D_{q,\ell} /K$	At depth zero, score all opened rows

The reported batch value averages the same 200 reference answers.

3 Deriving Optima

Minimize retrieval time at the required recall, within 32 GB RAM. Keep documents, queries and reference answers unchanged. Each method chooses its own index and search settings.

3.1 Original method

For each reference ID, record when its cluster is opened. Example: a document in the fifth cluster is included when $P \geq 5$. Write that rank as $s_{qi}^{(C)}$ for result i of query q . The smallest probe count meeting recall ρ is:

$$P_A(C, \rho) = \min \left\{ P : \frac{1}{n_q K} \sum_{q=1}^{n_q} \sum_{i=1}^K \mathbf{1}[s_{qi}^{(C)} \leq P] \geq \rho \right\}.$$

The indicator counts IDs whose clusters are open. At 99% recall, 200×100 reference IDs require at least 19,800 matches. Compute these ranks before timing. Try cluster counts 1, 256, 4096 and 8192. For each, test probe cutoffs for 80%, 90%, 95% and 99% recall, plus opening every cluster. Measure time and rows scored. The cutoffs apply to this query batch; they do not guarantee recall for new queries.

For a simple derivative, assume equal clusters, routing cost αC , row cost β , and fixed P :

$$T_A(C) \approx \alpha C + \beta NP/C, \quad \frac{dT_A}{dC} = \alpha - \frac{\beta NP}{C^2}, \quad C^* = \sqrt{\frac{\beta NP}{\alpha}}.$$

C^* balances cluster selection against document scanning. At a fixed recall target, $P_A(C, \rho)$ may change with C , so the measured comparison uses actual probe counts and cluster sizes.

3.2 Bitplanes

A 6400-document mask has 100 words. Removing 100 of 200 remaining candidates saves 100 scores, but adds 100 split-word visits. Let $W = \lceil n/64 \rceil$. If a split avoids ΔF scores, it helps when:

$$\Delta F a_B > Wb + \Delta T_{\text{other,B}}.$$

Left: saved scoring time. Right: split time and extra branch work. This assumes roughly constant cost per score; memory access and recall still need checking. If each cut halves one set, j cuts leave about $n2^{-j}$ rows. Holding other work fixed gives:

$$T_B(j) \approx T_0 + jWb + n2^{-j}a_B + T_{\text{other,B}}, \quad j^* = \log_2 \left(\frac{n \ln(2) a_B}{Wb} \right).$$

This estimates a cost balance, not recall. Identical bitplanes may remove no more rows; alternative branches add work. Measure both effects. Try every cluster/probe option from A. Test leaf size 128 with budgets 256 and 1024; leaf sizes 512 and N with unlimited work; and leaves 32 and 128 with budget 4096.

A node budget counts visited sets, including leaves. A leaf of size N allows scoring a cluster immediately. Record splits, final-mask reads and scores. A's probe cutoff is necessary for the same layout, but B's local limits may lose more IDs. Check the returned IDs against the recall target.

3.3 Backward Walk: settings to test

The new choices are starting prefix depth x and candidate target M , alongside cluster count C and probes P . All full scores still use 256 bits. A deeper start usually finds fewer rows, but may require more widening steps. A shallower start reduces lookups but may score many unnecessary rows.

For equal clusters and balanced independent bits, a first guess is $x \approx \log_2(Pn/M)$, since $Pn/2^x \approx M$. Round to an allowed depth, then check actual prefix counts. This is a starting point for experiments, not the optimum. Empty or uneven prefixes and the required recall can change the answer.

At fixed routing, widening helps recall by exposing more documents. Its cost is the extra range lookup work and the new rows scored; old rows must not be rescored. Compare saved full-score time with added prefix-lookup and bookkeeping time. The constants, final depth, recall and best settings remain unmeasured for this method.

$$\boxed{\text{saved scoring time} > \text{prefix lookup time} + \text{extra range work}}$$

3.4 Comparison

Let Θ_a be method a 's tested settings. The planned comparison chooses its lowest measured time subject to the recall and memory limits:

$$\theta_a^* = \underset{\theta \in \Theta_a}{\operatorname{argmin}} T_{a,\text{measured}}(\theta) \quad \text{subject to} \quad \overline{R}_a(\theta) \geq \rho, \quad M_{a,\text{process}}(\theta) \leq 32 \times 10^9 \text{ bytes.}$$

Method	Adjustable settings	Work that may be reduced
Original method	Cluster count C and probes P	Documents reaching the scorer
Bitplanes	C , P , leaf size and node budget	Scores, at the cost of mask and branch work
Backward Walk	C , P , starting depth x , candidate target M	Scores, at the cost of prefix lookups and new ranges

Compare methods after each chooses its own qualifying setting. Keep results below the recall target in the records, but do not count them as wins. This will identify the best observed setting in the tested range, not a global optimum. No such selection has been run for prefix Backward Walk yet.

4 Existing measurements and the next comparison

4.1 What the existing numbers measure

The previous experiment used one million MS MARCO document codes [5], 200 existing Nomic query vectors [4], and one exact top-100 reference set. Each method could choose its own cluster and search settings. The comparison used one CPU thread and a 32 GB process-memory limit on an Apple M5 Max. A is the cluster-and-scan baseline. B is the existing Bitplane search. C_{key} is the old weighted exact-key enumeration method.

C_{key} tries distinct keys in order of mismatch penalty. It does not widen 101101 to 10110*, so it is not the Backward Walk defined in this revision. The numbers below are retained unchanged as historical evidence. They must not be read as measurements of the new prefix method.

Table 1: Earlier results. C_{key} is the old method; prefix Backward Walk is not measured.

Required recall	Method	Actual recall (%)	Median (ms)
80%	A	80.01	0.5842
80%	B	80.01	0.5936
80%	C_{key}	80.01	0.6181
90%	A	90.02	1.43
90%	B	90.02	1.47
90%	C_{key}	90.02	1.55
95%	A	95.00	2.97
95%	B	95.00	3.07
95%	C_{key}	95.00	3.26
99%	A	99.00	8.74
99%	B	99.00	8.99
99%	C_{key}	99.00	9.73

At 99% recall, the selected A, B and C_{key} settings all scored about 295,858 documents per query. Their median times were 8.74, 8.99 and 9.73 ms. Those results show that the selected old alternatives did not avoid full scores. They do not tell us whether prefix relaxation can do better. Earlier source and PDF: [reports/search-math-experiments/](#). Earlier raw records: [results/fixed-data-comparison-2026-09-12/](#).

4.2 What remains to be measured

Implement the proposed prefix range lookup and widen one bit per level. Check that each document is scored at most once. Keep the document codes, query vectors and reference IDs byte-identical to the earlier benchmark. Build different index layouts from these same inputs.

Measure prefix depths, distinct rows scored, range lookups, recall, latency and memory. Do not convert an expected prefix count into assumed recall.

Vary clusters and probes for A, B and C. Vary B's branch settings and C's starting depth and candidate target. Include the missing larger-cluster cases. Compare each method's best qualifying setting at the same recall and RAM limits. Repeat timing measurements before drawing a conclusion.

5 Limitations and next steps

Prefix Backward Walk is a revised method definition. Its proposed layout and arithmetic examples are not implementation or benchmark results. Candidate-target stopping is approximate. A large returned set can still omit the most relevant reference documents. Balanced independent bits give useful count examples, but real embedding prefixes can be uneven and correlated, especially inside a cluster. Matryoshka motivates testing prefixes; it does not guarantee that every shorter sign prefix preserves recall.

Sorted-key lookup avoids rescoring old rows, but construction and updates still have costs. No insertion or speed advantage over HNSW has been measured.

6 Conclusion

The common observation is unchanged: query signs define an ideal binary pattern, and mismatches have costs determined by the query values. Bitplanes uses those values to narrow a set of documents while keeping alternative branches. Backward Walk starts from the query's prefix, removes its last bit when more candidates are needed, and scores only newly exposed rows.

The earlier key-enumeration experiment tested a different method. A new fixed-data comparison is required before saying where prefix Backward Walk wins or loses.

References

- [1] Exa Team. *How we built a web-scale vector database*. 2024. <https://exa.ai/blog/building-web-scale-vector-db>.
- [2] Y. Malkov and D. Yashunin. *Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs*. 2016; revised 2018. <https://arxiv.org/abs/1603.09320>.
- [3] A. Kusupati et al. *Matryoshka Representation Learning*. 2022. <https://arxiv.org/abs/2205.13147>.
- [4] Nomic AI. *nomic-embed-text-v1.5*. Model documentation. <https://huggingface.co/nomic-ai/nomic-embed-text-v1.5>.
- [5] Microsoft. *MS MARCO*. Passage collection and benchmark resources. <https://microsoft.github.io/msmarco/>.